

TP : Hadoop, HDFS, MapReduce, Spark

Partie 1 — TP Big Data : Compter les occurrences de mots avec **MapReduce** et **HDFS**

Objectif général

Mettre en place un petit cluster Hadoop (en mode pseudo-distribué) sur une machine locale, stocker un fichier texte dans le **système de fichiers distribué HDFS**, puis exécuter un programme **MapReduce** qui compte le nombre de fois où chaque mot apparaît dans ce fichier.

Ce TP permet de comprendre :

- le rôle de **HDFS** (Hadoop Distributed File System),
 - le fonctionnement du **paradigme MapReduce** (Map → Shuffle → Reduce),
 - et les bases de l'exécution d'un job Hadoop.
-

Étape 1 — Installation des prérequis

1. Installer Java

Hadoop fonctionne avec Java, donc il faut s'assurer qu'il est installé.

```
sudo apt update
sudo apt install openjdk-11-jdk -y
```

Vérifie :

```
java -version
```

Tu dois voir une version du type :

openjdk version "11.0.x"

⚠ Hadoop 3.x n'est pas compatible avec Java 17+, donc Java 11 est un bon choix.

2. Télécharger et installer Hadoop

Télécharge Hadoop depuis le site officiel Apache :

```
cd /usr/local
sudo wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
sudo tar -xvzf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 hadoop
```

3. Configurer les variables d'environnement

Ajoute ces lignes à la fin du fichier `~/.bashrc` :

```
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

Recharge les variables :

```
source ~/.bashrc
```

Vérifie :

```
hadoop version
```

Étape 2 — Configuration d'Hadoop

Hadoop doit savoir :

- où est Java,
- comment démarrer le **NameNode** et le **DataNode** (HDFS),

- et où stocker ses fichiers temporaires.

1 Fichier `hadoop-env.sh`

Ouvre :

```
nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

Cherche la ligne :

```
# export JAVA_HOME=
```

Et remplace-la par :

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

2 Fichier `core-site.xml`

```
nano $HADOOP_HOME/etc/hadoop/core-site.xml
```

Remplace le contenu par :

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

3 Fichier `hdfs-site.xml`

```
nano $HADOOP_HOME/etc/hadoop/hdfs-site.xml
```

Contenu :

```
<configuration>
  <property>
```

```
<name>dfs.replication</name>
<value>1</value>
</property>

<property>
  <name>dfs.namenode.name.dir</name>
  <value>file:/usr/local/hadoop_tmp/hdfs/namenode</value>
</property>

<property>
  <name>dfs.datanode.data.dir</name>
  <value>file:/usr/local/hadoop_tmp/hdfs/datanode</value>
</property>
</configuration>
```



Conseil pratique

Avant de lancer `start-dfs.sh`, crée ces répertoires et donne les bons droits :

```
sudo mkdir -p /usr/local/hadoop_tmp/hdfs/namenode
sudo mkdir -p /usr/local/hadoop_tmp/hdfs/datanode
sudo chown -R $USER:$USER /usr/local/hadoop_tmp
```

Ainsi, Hadoop aura les permissions nécessaires pour lire/écrire dans ces dossiers, et tu éviteras les erreurs comme :

| storage directory does not exist or is not accessible

🔧 Étape 3 — Formatage du NameNode

Avant de démarrer Hadoop pour la première fois, il faut **initialiser le système de fichiers HDFS** :

```
hdfs namenode -format
```

Si tu vois :

```
SHUTDOWN_MSG: Shutting down NameNode
```

✅ c'est bon !

⚙️ Étape 4 — Démarrage du cluster Hadoop

Lance :

```
start-dfs.sh
```

Puis vérifie les processus actifs :

```
jps
```

Tu dois voir :

```
NameNode  
DataNode  
SecondaryNameNode  
Jps
```

Et ouvre ton navigateur sur :

👉 <http://localhost:9870>

Tu verras l'interface du système HDFS.

📁 Étape 5 — Préparer les données d'entrée dans HDFS

1 Crée un répertoire d'entrée :

```
hdfs dfs -mkdir /input
```

2 Copie ton fichier texte local dans HDFS :

```
hdfs dfs -put ~/texte.txt /input
```

3 Vérifie :

```
hdfs dfs -ls /input
```



Étape 6 — Écrire et compiler le programme MapReduce

Voici la structure classique d'un programme `WordCount.java` :

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable>{
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values, Context conte
```

```

xt) throws IOException, InterruptedException {
    int sum = 0;
    for (IntWritable val : values) {
        sum += val.get();
    }
    result.set(sum);
    context.write(key, result);
}
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Compile et crée un jar :

```

javac -classpath $(hadoop classpath) -d wordcount_classes WordCount.java
jar -cvf wordcount.jar -C wordcount_classes/ .

```



Étape 7 — Exécuter le job MapReduce

```
hadoop jar wordcount.jar WordCount /input /output
```

Si tout est bien configuré, Hadoop :

1. lit ton fichier `/input/texte.txt` ,

2. découpe les lignes en mots (Map),
3. compte les occurrences (Reduce),
4. écrit le résultat dans `/output`.

Étape 8 — Consulter les résultats

```
hdfs dfs -cat /output/part-r-00000
```

Exemple de sortie :

```
big    2
data   3
hadoop 5
mapreduce 1
```

Étape 9 — Nettoyage ou relance du job

Si tu veux relancer le job :

```
hdfs dfs -rm -r /output
```

Puis relance :

```
hadoop jar wordcount.jar WordCount /input /output
```

Étape 10 — Erreurs fréquentes et leurs solutions

Erreur	Cause	Solution
<code>JAVA_HOME is not set</code>	Hadoop ne trouve pas Java	Vérifie <code>export JAVA_HOME</code> dans <code>hadoop-env.sh</code>
<code>InconsistentFSStateException</code>	NameNode mal initialisé	Supprimer <code>/tmp/hadoop-<user>/dfs</code> puis refaire <code>hdfs namenode -format</code>
<code>Input path does not exist</code>	Le dossier <code>/input</code> n'existe pas dans HDFS	Crée le dossier et charge ton fichier avec <code>hdfs dfs -put</code>

Erreur	Cause	Solution
Output directory already exists	/output existe déjà	Supprime avec <code>hdfs dfs -rm -r /output</code>
Permission denied	Mauvais droits sur les dossiers locaux Hadoop	Exécute avec le même utilisateur ou ajuste les permissions avec <code>chmod -R 755 /usr/local/hadoop_tmp</code>

Partie 2 — TP Big Data : Compter les mots avec **Apache Spark** et HDFS

Objectif

- Lire le fichier texte depuis **HDFS**.
- Compter le nombre de fois que chaque mot apparaît dans le fichier.
- Écrire le résultat dans **HDFS**.

Cette fois-ci, tu n'utiliseras pas MapReduce directement, mais **l'API Spark en Java ou Python** (ici, nous ferons Python pour simplicité).

Étape 1 — Installer Spark (mode standalone)

1. Télécharge Spark (compatible Hadoop 3.x) :

```
cd /usr/local
sudo wget https://downloads.apache.org/spark/spark-3.5.6/spark-3.5.6-bin-hadoop3.tgz
sudo tar -xvzf spark-3.5.6-bin-hadoop3.tgz
sudo mv spark-3.5.6-bin-hadoop3 spark
```

1. Configure les variables d'environnement dans `~/.bashrc` :

```
export SPARK_HOME=/usr/local/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
```

1. Recharge :

```
source ~/.bashrc
```

1. Vérifie :

```
spark-submit --version
```

Étape 2 — Vérifier que HDFS est actif

Avant de lancer Spark, assure-toi que Hadoop est actif :

```
start-dfs.sh  
jps
```

Tu dois voir :

```
NameNode  
DataNode  
SecondaryNameNode  
Jps
```

Et ton fichier doit déjà être dans `/input` sur HDFS :

```
hdfs dfs -ls /input
```

Étape 3 — Créer le script Spark pour le WordCount

Ici, on utilise **Python (PySpark)**, c'est simple et rapide.

Crée un fichier `wordcount_spark.py` :

```
from pyspark.sql import SparkSession  
  
# 1 Créer une session Spark  
spark = SparkSession.builder \  
    .appName("WordCount") \  
    .getOrCreate()
```

```
# 2 Lire le fichier depuis HDFS
text_file = spark.read.text("hdfs://localhost:9000/input/texte.txt").rdd

# 3 Compter les mots
counts = text_file.flatMap(lambda line: line[0].split()) \
    .map(lambda word: (word, 1)) \
    .reduceByKey(lambda a, b: a + b)

# 4 Sauvegarder le résultat dans HDFS
counts.saveAsTextFile("hdfs://localhost:9000/output_spark")

spark.stop()
```

Explications :

1. `flatMap` → découpe chaque ligne en mots.
2. `map` → associe chaque mot à la valeur 1.
3. `reduceByKey` → somme toutes les occurrences de chaque mot.
4. `saveAsTextFile` → écrit le résultat dans HDFS (`/output_spark`).

Étape 4 — Exécuter le job Spark

Si le dossier `/output_spark` existe déjà, supprime-le :

```
hdfs dfs -rm -r /output_spark
```

Puis lance le job Spark :

```
spark-submit wordcount_spark.py
```

Étape 5 — Vérifier les résultats

Affiche le contenu généré par Spark dans HDFS :

```
hdfs dfs -ls /output_spark
```

```
hdfs dfs -cat /output_spark/part-00000
```

Le format sera similaire à :

```
('bonjour', 3)
('Hadoop', 1)
('le', 1)
('monde', 1)
('Spark', 1)
```

⚡ Spark produit souvent des tuples (mot, compteur).

🧩 Étape 6 — Notes importantes

- Spark peut utiliser plusieurs partitions, donc tu peux avoir plusieurs fichiers `part-00000` , `part-00001` , etc.
- Spark est plus rapide que MapReduce sur de grands fichiers.
- Tout se fait en **mémoire**, mais HDFS est utilisé pour stocker l'entrée et la sortie.

Quand Spark crée plusieurs partitions, il produit **plusieurs fichiers** `part-00000` , `part-00001` , **etc.** dans le dossier de sortie HDFS. Chaque fichier contient une **partie du résultat**.

Voici comment gérer ça pour **voir tous les résultats** ou les **fusionner** :

1 Afficher toutes les partitions directement

Tu peux utiliser `hdfs dfs -cat` sur le dossier entier :

```
hdfs dfs -cat /output_spark/*
```

- Le va lire **tous les fichiers dans le dossier**, y compris `part-00000` , `part-00001` , etc.
- `_SUCCESS` est vide, donc ça n'affectera pas la sortie.

Résultat → tu verras **tous les mots et leurs occurrences** mélangés, exactement comme si c'était un seul fichier.

2 Fusionner toutes les partitions en un seul fichier

Si tu veux un **seul fichier final** pour ton rapport :

```
hdfs dfs -getmerge /output_spark /home/ramsey/output_spark_final.txt
```

- `getmerge` → récupère tous les fichiers `part-*` et les **concatène en un seul fichier local**.
- `/home/ramsey/output_spark_final.txt` → fichier local contenant **tous les résultats**.

Puis tu peux le vérifier :

```
cat /home/ramsey/output_spark_final.txt
```

3 Note importante

- Spark **parallélise automatiquement** selon le nombre de partitions.
- Sur un petit fichier comme ton TP, tu n'auras probablement qu'une partition (`part-00000`).
- Sur de gros fichiers, tu peux **forcer le nombre de partitions** si tu veux :

```
counts = counts.coalesce(1) # réduit à 1 partition pour sortie unique  
counts.saveAsTextFile("hdfs://localhost:9000/output_spark")
```

⚡ Cela force Spark à créer un seul fichier `part-00000`, pratique pour les TP ou rapports.